Github 账号：GlueNa2SiO3

个人博客关于密码学实验的链接：**https://github.com/Glue-Na2SiO3/GLUE**

实验题目：实验 **4**

实验摘要：

本次实验以 2016 年全国高校密码数学挑战赛赛题三为背景，针对 21 组 1024bit RSA 加密数据（格式为模数 `N`|加密指数 `e`|密文 `c`，密文由不超过 8 个明文字符填充后加密生成）展开破译研究。实验依次采用**共模攻击**、**低指数模数攻击**、**因数碰撞攻击**、**费马分解法**、**Pollard p-1 分解法**等多种密码攻击手段，逐步破解大部分加密帧，并通过明文片段的语义分析与拼接，还原出完整的明文内容。实验验证了 RSA 加密体制在使用低指数、重复模数、伪随机数生成器缺陷等场景下的安全漏洞，为 RSA 实际应用中的安全配置提供了实验依据。

题目描述

需破译 21 组 RSA 加密帧数据，每组数据包含三个 1024bit 的十六进制字段：模数 `N`、加密指数 `e`、密文 `c`（`c = m^e mod N`，`m` 为明文经填充后的结果）。已知明文本身为不超过 8 个字符的字符串，需通过多种 RSA 攻击方法还原各帧明文，并结合语义分析拼接出完整的原始信息。

过程（详细代码请参考 GitHub 仓库）

实验原理

1. 共模攻击

当同一明文使用**相同模数 N**、**不同且互质的公钥指数 e_1/e_2**加密时，攻击者可通过扩展欧几里得算法求解 `s` 和 `t` 满足 `e₁·s + e₂·t = 1`，再通过$m = (c_1{}^s \cdot c_2{}^t) \mathrm{mod} N$还原明文，无需分解模数。

2. 低指数模数攻击

若多个用户使用**相同低指数 e**（如 e=3、e=5）和不同模数 `N_i` 加密同一明文 `m`，且用户数量 `k ≥ e`，则可通过中国剩余定理（CRT）求解$x \equiv m^e \mathrm{mod}(N_1 \times N_2 \times \ldots \times N_k)$。当$m^e < \prod N_i$时，$x = m^e$，直接开 `e` 次方根即可得到明文。

3. 因数碰撞攻击

若两个模数 N_1和 N_2因伪随机数生成器缺陷共享质因子 `p`，则通过 `gcd(N₁, N₂) = p` 可快速分解两个模

数，进而求解私钥破解密文。

4. 费马分解法

针对大整数 `N = p×q`（`p` 和 `q` 差值较小），通过寻找 `x` 使得 `x² - N = y²`，则 `N = (x-y)(x+y)`，适用于因子差值较小的模数分解。

5. Pollard p-1 分解法

若 `N` 的质因子 `p` 满足 `p-1` 为**平滑数**（质因数均不超过边界 `B`），则通过计算$a = 2^M \bmod N$（`M` 为不超过 `B` 的质数幂乘积），再求 `gcd(a-1, N)`可得到 `p`，适用于特定结构的大整数分解。

辅助函数定义

实现十六进制与整数、文本的转换，以及扩展欧几里得算法、中国剩余定理的核心逻辑

```
import gmpy2
import math
from collections import defaultdict
import time

# 十六进制转整数
def hex_str_to_int(hex_str):
    return int(hex_str, 16)

# 整数转十六进制字符串（去除 0x 前缀）
def int_to_hex_str(num):
    return hex(num)[2:] if num != 0 else "0"

# 十六进制字符串转明文（取最后 16 位，UTF-8 解码）
def hex_to_plain_text(hex_str):
    try:
        bytes_data = bytes.fromhex(hex_str[-16:])
        return bytes_data.decode('utf-8', errors='ignore')
    except Exception as e:
```

```
        return f"解码错误: {e}"

# 扩展欧几里得算法
def extended_euclidean(a, b):
    if a == 0:
        return b, 0, 1
    else:
        gcd, x, y = extended_euclidean(b % a, a)
        return gcd, y - (b // a) * x, x

# 中国剩余定理实现
def chinese_remainder_theorem(congruences):
    total_mod = 1
    for _, mod in congruences:
        total_mod *= mod

    result = 0
    for rem, mod in congruences:
        mod_i = total_mod // mod
        gcd, inv, _ = extended_euclidean(mod_i, mod)
        if gcd != 1:
            raise ValueError("模数不互质，无法应用中国剩余定理")
        result += rem * mod_i * inv

    return result % total_mod, total_mod
```

数据预处理

读取 21 组 RSA 加密帧数据，提取模数、指数、密文并存储为列表，优化原代码的文件读取逻辑，避免硬编码路径：

```
def read_rsa_frame_data(frame_dir="rsa_frames"):
```

```
    """读取 RSA 加密帧数据，返回模数、指数、密文列表（十六进制）
"""
    moduli_list = []  # 模数 N 列表
    exponents_list = []# 加密指数 e 列表
    ciphers_list = []  # 密文 c 列表

    for frame_idx in range(21):
        file_path = f"{frame_dir}/Frame{frame_idx}"
        try:
            with open(file_path, "r", encoding="utf-8") as f:
                frame_data = f.read().strip()
                # 按 1024bit（256 个十六进制字符）分割
                mod_hex = frame_data[0:256]
                exp_hex = frame_data[256:512]
                cipher_hex = frame_data[512:768]

                moduli_list.append(mod_hex)
                exponents_list.append(exp_hex)
                ciphers_list.append(cipher_hex)
        except FileNotFoundError:
            print(f"警告：未找到 Frame{frame_idx}文件")
            moduli_list.append("")
            exponents_list.append("")
            ciphers_list.append("")

    return moduli_list, exponents_list, ciphers_list
```

重复值与公共因子检测

优化原有的重复值查找逻辑，减少冗余循环；新增公共因子检测函数，为共模攻击、因数碰撞攻击提供依据

```
    def detect_duplicate_items(data_list):
        """检测列表中的重复元素，返回{元素：对应的索引集合}"""
        duplicate_map = defaultdict(set)
        for idx, item in enumerate(data_list):
            if item == "":
                continue
            duplicate_map[item].add(idx)
        # 过滤仅出现一次的元素
        duplicate_map = {k: v for k, v in duplicate_map.items() if len(v) >= 2}
        return duplicate_map


    def find_common_prime_factors(hex_moduli):
        """检测模数间的公共质因子，返回{质因子：对应的帧索引集合}"""
        factor_map = defaultdict(set)
        int_moduli = [hex_str_to_int(m) for m in hex_moduli if m != ""]
        frame_indices = [i for i, m in enumerate(hex_moduli) if m != ""]

        for i in range(len(int_moduli)):
            for j in range(i + 1, len(int_moduli)):
                mod1 = int_moduli[i]
                mod2 = int_moduli[j]
                common_factor = math.gcd(mod1, mod2)
                if common_factor > 1 and common_factor < mod1 and common_factor < mod2:
                    factor_map[common_factor].add(frame_indices[i])
                    factor_map[common_factor].add(frame_indices[j])
```

```
        return factor_map

    # 检测重复模数（共模攻击依据）
    duplicate_mods = detect_duplicate_items(moduli)
    print("重复模数对应的帧索引：", duplicate_mods)

    # 检测重复指数（低指数攻击依据）
    duplicate_exps = detect_duplicate_items(exponents)
    print("重复指数对应的帧索引：", duplicate_exps)

    # 检测模数公共因子（因数碰撞攻击依据）
    common_factors = find_common_prime_factors(moduli)
    print("模数公共因子对应的帧索引：", common_factors)

    # 低指数检测（按指数大小排序）
    exp_with_idx = [(i, hex_str_to_int(e)) for i, e in
enumerate(exponents) if e != ""]
    sorted_exps = sorted(exp_with_idx, key=lambda x: x[1])
    print("按指数从小到大排序的帧索引与指数值：", sorted_exps)
```

共模攻击

修改原攻击函数，优化负指数处理逻辑，针对检测到的重复模数帧（Frame0、Frame4）进行破译：

```
    def rsa_common_modulus_attack(mod_hex, exp1_hex, exp2_hex,
cipher1_hex, cipher2_hex):
        """共模攻击：同一模数不同指数的 RSA 破解"""
        n = hex_str_to_int(mod_hex)
        e1 = hex_str_to_int(exp1_hex)
        e2 = hex_str_to_int(exp2_hex)
```

```
    c1 = hex_str_to_int(cipher1_hex)
    c2 = hex_str_to_int(cipher2_hex)

    # 扩展欧几里得求 s 和 t，满足 e1*s + e2*t = 1
    gcd, s, t = extended_euclidean(e1, e2)
    if gcd != 1:
        raise ValueError("两个指数不互质，无法执行共模攻击")

    # 处理负指数（求模逆元）
    if s < 0:
        c1 = gmpy2.invert(c1, n)
        s = -s
    if t < 0:
        c2 = gmpy2.invert(c2, n)
        t = -t

    # 计算明文
    m_int = (pow(c1, s, n) * pow(c2, t, n)) % n
    m_hex = int_to_hex_str(m_int)
    return hex_to_plain_text(m_hex)

# 对 Frame0 和 Frame4 执行共模攻击
if duplicate_mods:
    target_mod = list(duplicate_mods.keys())[0]
    target_frames = list(duplicate_mods[target_mod])
    frame0, frame4 = target_frames[0], target_frames[1]
    common_mod_result = rsa_common_modulus_attack(
        moduli[frame0], exponents[frame0], exponents[frame4],
        ciphers[frame0], ciphers[frame4]
```

```
        )
        print(f"Frame{frame0} 与 Frame{frame4} 共模攻击结果：", common_mod_result)
```

低指数攻击（结合 CRT）

针对低指数帧（Frame3、8、12、16、20 为 e=5；Frame7、11、15 为 e=3），通过中国剩余定理求解后开方得到明文：

```
    def rsa_low_exponent_attack(frame_indices, moduli, exponents, ciphers, exp):
        """低指数攻击：结合中国剩余定理破解"""
        congruence_list = []
        for idx in frame_indices:
            if moduli[idx] == "" or ciphers[idx] == "":
                continue
            c = hex_str_to_int(ciphers[idx])
            n = hex_str_to_int(moduli[idx])
            congruence_list.append((c, n))

        # 应用中国剩余定理
        crt_val, total_mod = chinese_remainder_theorem(congruence_list)
        # 开 e 次方根
        m_int = gmpy2.iroot(gmpy2.mpz(crt_val), exp)[0]
        m_hex = int_to_hex_str(m_int)
        return hex_to_plain_text(m_hex)

    # 对 e=5 的帧（3,8,12,16,20）执行低指数攻击
    exp5_frames = [3,8,12,16,20]
```

```
    low_exp5_result = rsa_low_exponent_attack(exp5_frames, moduli, exponents, ciphers, 5)
    print(f"e=5 低指数攻击结果（Frame{exp5_frames}）：", low_exp5_result)

    # 对e=3的帧（7,11,15）执行低指数攻击（需后续结合其他方法）
    exp3_frames = [7,11,15]
    low_exp3_result = rsa_low_exponent_attack(exp3_frames, moduli, exponents, ciphers, 3)
    print(f"e=3 低指数攻击结果（Frame{exp3_frames}）：", low_exp3_result)
```

因数分解攻击

实现费马分解法和带超时的 Pollard p-1 分解法，用于分解模数并求解私钥：

```
    def fermat_factorize(n):
        """费马分解法：分解大整数N=pq"""
        if n % 2 == 0:
            return 2, n // 2
        # 检查是否为完全平方数
        root = math.isqrt(n)
        if root * root == n:
            return root, root
        # 费马核心循环
        x = root + 1
        max_iter = 10**6  # 迭代上限
        for _ in range(max_iter):
            x_sq = x * x
            y_sq = x_sq - n
```

```
        y = math.isqrt(y_sq)
        if y * y == y_sq:
            p = x + y
            q = x - y
            if p * q == n:
                return p, q
        x += 1
    return None, None

def pollard_p1_factorize(n, timeout=4):
    """Pollard p-1 分解法（带超时）"""
    start_time = time.time()
    if n <= 3:
        return None
    if n % 2 == 0:
        return 2
    # 小质数预检测
    small_primes = [2,3,5,7,11,13,17,19,23,29]
    for p in small_primes:
        if n % p == 0:
            return p
    # 尝试不同基值和边界
    bases = [2,3,5,7,11]
    boundaries = [1000,5000,10000]
    for base in bases:
        for B in boundaries:
            if time.time() - start_time > timeout:
                return None
            a = base
```

```
            for i in range(2, B+1):
                a = pow(a, i, n)
                if i % 1000 == 0:
                    g = math.gcd(a-1, n)
                    if 1 < g < n:
                        return g
            g = math.gcd(a-1, n)
            if 1 < g < n:
                return g
    return None

# 遍历模数执行因数分解
def factorize_moduli(moduli):
    """遍历模数并尝试分解"""
    factor_results = {}
    for idx, mod_hex in enumerate(moduli):
        if mod_hex == "":
            continue
        n = hex_str_to_int(mod_hex)
        # 先尝试费马分解
        p, q = fermat_factorize(n)
        if p and q:
            factor_results[idx] = (p, q)
            continue
        # 再尝试 Pollard p-1 分解
        p = pollard_p1_factorize(n)
        if p:
            q = n // p
            factor_results[idx] = (p, q)
```

```
        return factor_results

    # 执行模数分解
    mod_factors = factorize_moduli(moduli)
    print("可分解的模数及因子：", mod_factors)

    # 通过分解的因子求解私钥并解密
    def rsa_decrypt_by_factor(p, q, mod_hex, exp_hex, cipher_hex):
        """通过质因子求解私钥并解密"""
        n = hex_str_to_int(mod_hex)
        e = hex_str_to_int(exp_hex)
        c = hex_str_to_int(cipher_hex)
        phi_n = (p-1) * (q-1)
        d = gmpy2.invert(e, phi_n)  # 求私钥 d
        m_int = gmpy2.powmod(c, d, n)  # 解密
        m_hex = int_to_hex_str(m_int)
        return hex_to_plain_text(m_hex)

    # 对 Frame10（费马分解成功）解密
    if 10 in mod_factors:
        p10, q10 = mod_factors[10]
        frame10_result = rsa_decrypt_by_factor(p10, q10, moduli[10],
exponents[10], ciphers[10])
        print("Frame10 解密结果：", frame10_result)
```

因数碰撞攻击

针对存在公共因子的模数帧（Frame1、Frame18），通过公共因子分解模数后解密

```
    def rsa_factor_collision_attack(moduli, exponents, ciphers, common_factors):
        """因数碰撞攻击：利用模数公共因子解密"""
        decrypt_results = {}
        for factor, frame_indices in common_factors.items():
            for idx in frame_indices:
                mod_hex = moduli[idx]
                if mod_hex == "":
                    continue
                n = hex_str_to_int(mod_hex)
                p = factor
                q = n // p
                # 解密
                result = rsa_decrypt_by_factor(p, q, mod_hex, exponents[idx], ciphers[idx])
                decrypt_results[idx] = result
        return decrypt_results


    # 执行因数碰撞攻击
    collision_results = rsa_factor_collision_attack(moduli, exponents, ciphers, common_factors)
    print("因数碰撞攻击结果：", collision_results)
```

明文拼接与补全

通过已破解的明文片段，结合语义分析与名人名言知识库，补全剩余明文。已破解的片段包括：**My secre**、**t is a f**、**Logic**、**will get**、**Imagin**、**m A to B**、**instein** 等，最终拼接得到完整明文：

My secret is a famous saying of Albert Einstein. That is "Logic will get you from A to B. Imagination will take you everywhere."

本次实验成功破解 21 组 RSA 加密帧中的 13 组，剩余 8 组通过语义分析补全明文。各帧的破解方法与结果如下表所示：

帧编号	破解方法	明文片段
0、4	共模攻击	My secre
3、8、12、16、20	低指数攻击（e=5）	t is a f
10	费马分解法	will get
2	Pollard p-1 分解法	That is
6	Pollard p-1 分解法	Logic
19	Pollard p-1 分解法	instein
1、18	因数碰撞攻击	Imagin / m A to B
7、11、15	低指数攻击（e=3，结合语义补全）	ation will take you everywhere

总结（完成心得与其它，主要自己碰到的问题和解决问题的方法）

RSA 安全漏洞验证：实验证实 **RSA** 加密体制在使用重复模数、低加密指数、弱伪随机数生成器（导致模数共享质因子） 时存在严重安全漏洞，攻击者可通过共模攻击、低指数攻击、因数碰撞攻击等手段快速破译，无需分解大整数。

代码优化问题：实验中遇到十六进制数据前缀 **0x** 导致的解析错误，通过切片去除前缀解决；同时优化了原代码的双重循环逻辑，减少了重复比较，提升了数据检测效率。

攻击方法选择：破解 **RSA** 时应优先尝试共模攻击和因数碰撞攻击（计算成本低），再尝试低指数攻击，最后使用因数分解类攻击（计算成本高）。

参考文献（包括参考的书籍，论文，**URL** 等，很重要）

全国高校密码数学挑战赛（**2016**）赛题解析：**https://www.cryptographychallenge.org**

《密码学原理与实践》（**Douglas R. Stinson** 著）

Pollard p-1 分解法原理论文：Pollard J. M. Theorems on factorization and primality testing [J]. Mathematical Proceedings of the Cambridge Philosophical Society, 1975.